

Unified Atlas + MoO Enterprise AI Operations Platform Master PRD

Status: Architecture PRD, implementation-ready reference

Date: 2026-06-27

Primary decision: launch on OAuth 2.0 scopes, short-lived JWT delegation, PostgreSQL RLS, sandboxed tools, and append-only audit.

Future decision: keep Lean specification and ZK proof fields as dormant upgrade hooks, not launch blockers.

1. EXECUTIVE SUMMARY, SYSTEM THESIS, & TARGET PERSONAS

1.1 Core Definition

Atlas + MoO is a zero-trust AI operations platform that wraps frontier LLMs in enterprise execution controls so they can plan, patch repositories, query data, run sandboxes, and produce artifacts without receiving implicit trust. Atlas owns the semantic data substrate; MoO owns runtime orchestration; OAuth/JWT/RLS/tool sandboxing enforce the execution boundary.

The system transforms untrusted model intent into governed enterprise action:

```
User intent
-> GoalContract
-> MetaOrchestrationRun
-> scoped OrchestratorRuns and AgentSessions
-> JWT-gated ToolCalls
-> PostgreSQL RLS-filtered data access
-> sandboxed execution
-> verified NextActionDecision
-> append-only AuditEvent ledger
```

1.2 Target Personas

Persona	Primary Jobs	Current Friction	Product Promise
Enterprise AI Platform Team	Provide safe AI execution infrastructure across engineering, security, support, and operations	Uncontrolled agent tools, unclear ownership boundaries, no standard delegation model, fragmented audit	Centralized AI control plane with tenant/workspace isolation, scoped delegation, tool routing, cost controls, and audit
Autonomous SecOps Engineer	Investigate alerts, query logs, open patches, run sandboxes, prepare incident evidence	Manual triage loops, secrets and production blast-radius concerns, weak evidence chains	Agents can investigate and prepare changes autonomously while irreversible actions remain gated
Principal Software Architect	Convert high-level goals into implementation plans, dependency graphs, code patches, and review packets	Context loss, unreviewed generated code, hidden tool activity, noisy approvals	Transparent goal contracts, strategy inspection, SYSTEM_DESIGN.md planning artifacts, test-backed PR loops

1.3 Core Transformation Loop

The platform treats every LLM and agent process as untrusted. Authority is conveyed only through short-lived delegation and database-enforced scope.

- Human or integration submits intent.
- API Gateway authenticates OAuth principal and resolves tenant/workspace.
- Goal Contract Engine records allowed actions, blocked actions, constraints, risk class, and budget.
- Meta-Orchestrator selects fast path or blended orchestration.
- Identity Service issues short-lived JWT Delegation Token to an AgentSession.
- Agent attempts ToolCall through Tool Router, never directly against data or infrastructure.
- Tool Router verifies JWT signature, audience, expiry, scopes, resource allowlist, and

- classification ceiling.
- 8. API layer sets app.current_tenant_id, app.current_workspace_id, app.current_actor_id in the database transaction.
- 9. PostgreSQL RLS filters all reads and rejects invalid writes.
- 10. Tool input is schema-validated; execution occurs in sandbox; output is schema-validated.
- 11. Derived artifacts inherit parent classifications.
- 12. Audit Service writes immutable events for requests, policy checks, tool calls, decisions, and mutations.

1.4 Plane Separation

Plane	Canonical Records	Authority	Consumer-Facing Abstraction
Atlas Data Plane	object_instances, link_instances, tasks, artifacts, classifications	Defines enterprise state and graph meaning	Project Cockpit, Graph Explorer
MoO Control Plane	goal_contracts, meta_orchestration_runs, orchestrator_runs, agent_sessions, next_action_candidates, next_action_decisions	Coordinates execution, not final authority	Strategy Inspector, Run Trace Viewer
Identity Plane	tenants, workspaces, users, service_accounts, OAuth clients, delegation tokens	Grants scoped access	Admin Console
Tool Plane	tool_definitions, tool_calls, tool_results, sandbox_profiles	Executes only after scope and policy checks	Tool Timeline
Audit Plane	audit_logs, hash chain fields, policy result records	Accountability and replay	Audit Trail
Future Proof Plane	formal_rules, lean_specs, zk_predicates, proof_artifacts	Dormant launch fields for later assurance	Compliance Evidence

2. CORE SYSTEM USER INTERFACES & PRODUCT SURFACES

2.1 Project Cockpit & Graph Explorer

The Project Cockpit exposes Atlas state, not raw agent internals.

Required capabilities:

Capability	Backing Tables	Orchestrator Integration	Display Rule
Task graph	object_instances, link_instances, tasks	Planner Orchestrator writes candidate task graph	Show operational tasks as actionable; candidate tasks as proposed
Blocker map	link_instances(link_type='blockers')	Workflow Orchestrator validates blocked states	Never allow MoO to bypass a blocked Atlas edge
Semantic metadata	object_instances.properties, classification	Memory and Research Orchestrators read under scope	Redact fields above viewer classification ceiling
Artifact trail	object_instances(object_type='Artifact'), audit_logs	Coding/Verification Orchestrators produce artifacts	Show source links and verification status

Display paradigm:

```
cockpit_panels:
  left: project_graph_and_filters
  center: selected_task_or_artifact_detail
  right: blockers_evidence_recent_runs
  bottom: audit_and_tool_timeline
```

2.2 Goal Contract Editor & Strategy Inspector

The Goal Contract Editor is the human-controllable contract between intent and automation.

Required fields:

```

goal_contract_editor_fields:
  objective: string
  constraints: string[]
  allowed_actions: string[]
  blocked_actions: string[]
  acceptance_criteria: string[]
  risk_class: low | medium | high | unacceptable
  budget:
    max_cost_usd: number
    max_wall_clock_seconds: integer
    max_tool_calls: integer
    max_agent_sessions: integer
  approval_boundaries:
    - external_side_effect
    - protected_branch_merge
    - production_deploy
    - public_export
    - permission_change
    - restricted_data_access

```

Strategy Inspector must show:

Field	Source	Requirement
Selected strategy	meta_orchestration_runs.selected_strategy	Persist before worker agents start
Orchestrator profiles	orchestrator_runs.profile	Show Planner, Research, Evidence, Workflow, Tool, Coding, Critic, Safety-Verification
Model routing metrics	orchestrator_runs.input_tokens, output_tokens, cost_usd, latency_ms	Summarize without exposing raw prompts by default
Rejection reasons	strategy_selection.rejected_candidates	Explain why fast-path or debate was skipped
Human gates	goal_contracts.approval_boundaries	Show approval condition and responsible human group

2.3 Approval Inbox & Run Trace Viewer

The Approval Inbox should interrupt humans only at consequential boundaries.

Approval card required content:

```

approval_card:
  proposed_action: string
  target_resources: string[]
  risk_class: low | medium | high
  reversibility_class: reversible | compensatable | irreversible
  scopes_requested: string[]
  files_or_objects_touched: string[]
  tests_or_verifications: string[]
  critic_findings: string[]
  rollback_or_compensation_plan: string
  audit_trace_id: string

```

Run Trace Viewer required timeline:

```

GoalContract.created
StrategySelection.persisted
DelegationToken.issued
AgentSession.started
ToolCall.requested
PermissionCheck.allowed_or_denied
SandboxRun.completed
VerificationRun.completed
NextActionDecision.selected
AuditEvent.appended

```

2.4 Local IDE Transparency File Rules

For non-trivial code execution, the platform writes a planning artifact into the workspace before agent file mutation.

```

SYSTEM_DESIGN.md_required_when:
  - code_change_risk >= medium
  - touches_core_system: true
  - multi_file_refactor: true

```

- production_or_customer_data_impact: true
- human_review_required: true

SYSTEM_DESIGN.md must include:

```
system_design_sections:
- goal_contract_id
- meta_orchestration_run_id
- intended_files
- prohibited_files
- allowed_tools
- expected_patch_shape
- acceptance_criteria
- verification_commands
- rollback_plan
```

Fast-path exception:

```
SYSTEM_DESIGN.md_skipped_when_all:
- risk_class: low
- action_reversibility: reversible
- external_side_effect: false
- production_effect: false
- estimated_cost_usd_lte: 0.05
```

3. STRATEGY SELECTION & ENGINE LIFE CYCLES

3.1 MoO Orchestrator Profiles

Profile	Primary Domains	Capabilities	Required Tools	Output Records
Planner	project planning, task graphing, dependency analysis	decomposes goals, assigns subtasks, builds blocker graph	ontology:read, task:write	NextActionCandidate, LinkInstance(blocks)
Research	source retrieval, market/technical research	fetches permitted sources, summarizes evidence	web:read, ontology:read, artifact:write	Artifact, Statement, Evidence
Evidence	claim support, citation checks, contradiction detection	validates claims and links evidence	ontology:read, artifact:read, evidence:write	VerificationRun, EvidenceCheck
Workflow	state machines, approvals, scheduled automations	evaluates guards, transitions workflow nodes, pauses for approvals	workflow:read, workflow:write, approval:request	WorkflowRun, ApprovalRequest
Tool	tool state, sandbox routing, file/db/shell/calendar orchestration	picks tools, validates inputs, tracks permissions	tool:call, sandbox:execute	ToolCall, ToolCallResult
Coding	repository patches, tests, PR preparation	reads code, writes patches, generates review packets	repo:read, patch:write, sandbox:execute, pr:create	PatchArtifact, PullRequestArtifact
Critic	red-team review, edge cases, regression risk	challenges assumptions, blocks unsafe changes, requests rework	artifact:read, repo:read, verification:write	CriticFinding, VerificationCheck
Safety-Verification	policy validation, test execution, sandbox verification	runs test suites, validates acceptance criteria, enforces stop rules	sandbox:execute, test:run, audit:read	VerificationRun, NextActionDecision

3.2 Strategy Selection Lifecycle

```
strategy_selection_order:
- create_or_update_goal_contract
- classify_task_class_and_risk
- evaluate_low_risk_fast_path
- resolve_required_scopes
- resolve_current_workspace_graph_snapshot
- score_orchestrator_profiles
- select_strategy_mode
- persist_strategy_selection
- issue_delegation_tokens
- start_orchestrator_runs
```

Strategy modes:

Mode	Trigger	Orchestrators	Human State
Fast-path	Low-risk, reversible, local	Tool only or Planner + Tool	No approval
Single orchestrator	Low-medium risk, simple domain	One selected profile	No approval unless boundary triggered
Sequential	Evidence must precede action	Research -> Evidence -> Planner -> Tool	Possible confirmation
Blended	Code, architecture, security, or core system change	Coding + Critic + Safety-Verification	Review-ready notification
Human-gated	Irreversible, production, financial, public, permission change	Planner + Safety + Human Approval	Explicit approval
Refuse/stop	Blocked or unacceptable	Meta-Orchestrator only	Notify owner/security

3.3 Low-Risk Fast-Path Protocol

All criteria must be true:

```
low_risk_fast_path_required_all:
  goal_contract.risk_class: low
  action.reversibility_class: reversible
  external_side_effect: false
  production_effect: false
  classification_max: internal
  protected_resource_touched: false
  required_scopes_present: true
  jwt_ttl_minutes_lte: 15
  rls_context_valid: true
  tool_input_schema_valid: true
  rollback_snapshot_available: true
  prior_conflict: false
  critic_required: false
  estimated_cost_usd_lte: 0.05
  estimated_latency_seconds_lte: 30
```

Allowed fast-path actions:

```
fast_path_allowed_actions:
  - create_candidate_task
  - update_non_sensitive_task_metadata
  - create_internal_draft_artifact
  - run_sandboxed_unit_test
  - local_branch_scaffolding
  - format_or_refactor_non_core_local_code
  - recompute_cached_workspace_projection
```

Denied fast-path actions:

```
fast_path_denied_actions:
  - merge_to_protected_branch
  - production_deployment
  - public_export
  - permission_or_policy_change
  - secret_or_key_access
  - destructive_delete
  - restricted_data_read
  - database_schema_migration_in_production
```

3.4 Anti-Fatigue Filter Matrix

Action Class	Examples	Infrastructure Decision	Human Notification
Reversible local	sandbox test, draft artifact, local patch before PR	Auto-allow if scope/RLS pass	None
Reversible reviewable	private PR draft, non-core code refactor	Auto-complete, notify at review-ready state	One bundled notification
Compensatable	external ticket update, non-production integration writeback	Ask confirmation or batch approval	One confirmation
Irreversible	protected branch merge, production deploy, public export, permission change	Block until approval record exists	Immediate approval request
Unacceptable	cross-tenant access, policy bypass, secret exfiltration	Refuse/stop and audit	Security/owner alert

Reversibility engine:

```
reversible_if_all:
- before_snapshot_exists
- rollback_action_type_exists
- no_external_system_commit
- no_public_visibility_change
- no_secret_or_permission_change
- no_protected_branch_update

irreversible_if_any:
- production_deploy
- merge_to_protected_branch
- public_export
- confidential_data_disclosure
- permission_policy_change
- secret_or_key_rotation
- destructive_delete_without_restore
```

4. PRAGMATIC ZERO-TRUST TOKEN & ROUTING BLUEPRINTS

4.1 JWT Delegation Token Schema

```
{
  "$id": "AgentSessionDelegationJWTClaims",
  "type": "object",
  "required": [
    "iss",
    "sub",
    "aud",
    "exp",
    "iat",
    "jti",
    "tenant_id",
    "workspace_id",
    "session_id",
    "scopes"
  ],
  "properties": {
    "iss": {"const": "atlas-identity-service"},
    "sub": {"type": "string", "description": "agent_identity_id"},
    "aud": {"const": "atlas-tool-router"},
    "exp": {"type": "integer", "description": "maximum TTL 15 minutes"},
    "iat": {"type": "integer"},
    "nbf": {"type": "integer"},
    "jti": {"type": "string"},
    "tenant_id": {"type": "string"},
    "workspace_id": {"type": "string"},
    "session_id": {"type": "string"},
    "meta_run_id": {"type": "string"},
    "orchestrator_run_id": {"type": "string"},
    "scopes": {
      "type": "array",
      "items": {
        "enum": [
          "ontology:read",
          "ontology:write",
          "task:read",
          "task:write",
          "artifact:read",
          "artifact:write",
          "repo:read",
          "patch:write",
          "sandbox:execute",
          "test:run",
          "pr:create",
          "workflow:trigger",
          "approval:request"
        ]
      }
    },
    "resource_allowlist": {"type": "array", "items": {"type": "string"}},
    "path_allowlist": {"type": "array", "items": {"type": "string"}},
    "tool_allowlist": {"type": "array", "items": {"type": "string"}},
    "max_classification": {"enum": ["public", "internal", "confidential", "restricted"]},
    "max_tool_calls": {"type": "integer"},
    "max_cost_usd": {"type": "number"}
  }
}
```

Token rules:

```

delegation_token_rules:
  max_ttl_minutes: 15
  refresh_requires_parent_session: true
  scopes_must_be_subset_of_parent: true
  agents_cannot_request_audit_append_scope: true
  audit_events_written_by_gateway_or_audit_service_only: true
  jti_replay_cache_required: true

```

4.2 ToolCall Verification Pipeline

```

tool_call_verification_pipeline:
- receive_tool_call:
  owner: API Gateway
  input: Authorization bearer JWT + ToolCall payload
  mutation: audit_logs(event_type='tool_call.received')
- verify_token:
  owner: Identity Service
  checks:
    - JWKS signature valid
    - iss == atlas-identity-service
    - aud == atlas-tool-router
    - exp > now
    - nbf <= now
    - jti unused
- resolve_context:
  owner: API Gateway
  values:
    - tenant_id
    - workspace_id
    - actor_id
    - session_id
    - meta_run_id
    - orchestrator_run_id
- set_db_session:
  owner: Database Access Layer
  statements:
    - SET LOCAL app.current_tenant_id
    - SET LOCAL app.current_workspace_id
    - SET LOCAL app.current_actor_id
    - SET LOCAL app.current_session_id
- load_runtime_records:
  owner: MoO Runtime Service
  tables:
    - agent_sessions
    - orchestrator_runs
    - meta_orchestration_runs
  enforced_by: PostgreSQL RLS
- authorize_operation:
  owner: Tool Router
  checks:
    - required_tool_scope subset JWT.scopes
    - resource in resource_allowlist or object_set_allowlist
    - file path in path_allowlist
    - resource.classification <= JWT.max_classification
    - tool_call_count <= max_tool_calls
    - cost <= max_cost_usd
- validate_input_schema:
  owner: Tool Router
  source: tool_definitions.input_schema
- execute_sandbox:
  owner: Tool Router
  boundary: isolated sandbox profile
- validate_output_schema:
  owner: Tool Router
  source: tool_definitions.output_schema
- classify_output:
  owner: Classification Engine
  rule: output_classification = max(parent classifications)
- persist_results:
  owner: Atlas Data Plane or MoO Control Plane
  tables:
    - object_instances
    - tool_calls
    - audit_logs
- emit_completion_audit:
  owner: Audit Service
  mutation: audit_logs(event_type='tool_call.completed')

```

5. INFRASTRUCTURE DATA PROTECTION (POSTGRESQL RLS SCHEMAS)

5.1 Core RLS Session Contract

Every database transaction from API Gateway or Tool Router must set:

```
SET LOCAL app.current_tenant_id = '<tenant_uuid>';
SET LOCAL app.current_workspace_id = '<workspace_uuid>';
SET LOCAL app.current_actor_id = '<actor_uuid>';
SET LOCAL app.current_session_id = '<agent_session_uuid_or_human_session_uuid>';
```

No service account may connect with table-owner privileges for normal reads/writes. Migration/admin roles are physically separate from runtime roles.

5.2 Table: object_instances

```
CREATE TABLE object_instances (
  id uuid PRIMARY KEY,
  tenant_id uuid NOT NULL,
  workspace_id uuid NOT NULL,
  object_type text NOT NULL,
  properties jsonb NOT NULL DEFAULT '{}'::jsonb,
  classification text NOT NULL CHECK (
    classification IN ('public', 'internal', 'confidential', 'restricted')
  ),
  source_object_ids uuid[] NOT NULL DEFAULT ARRAY[]::uuid[],
  created_by uuid NOT NULL,
  updated_at timestamptz NOT NULL DEFAULT now()
);

ALTER TABLE object_instances ENABLE ROW LEVEL SECURITY;
ALTER TABLE object_instances FORCE ROW LEVEL SECURITY;

CREATE POLICY object_instances_tenant_workspace_select ON object_instances
FOR SELECT
USING (
  tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::uuid
  AND workspace_id = NULLIF(current_setting('app.current_workspace_id', true), '')::uuid
);

CREATE POLICY object_instances_tenant_workspace_write ON object_instances
FOR INSERT WITH CHECK (
  tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::uuid
  AND workspace_id = NULLIF(current_setting('app.current_workspace_id', true), '')::uuid
);

CREATE POLICY object_instances_tenant_workspace_update ON object_instances
FOR UPDATE
USING (
  tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::uuid
  AND workspace_id = NULLIF(current_setting('app.current_workspace_id', true), '')::uuid
)
WITH CHECK (
  tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::uuid
  AND workspace_id = NULLIF(current_setting('app.current_workspace_id', true), '')::uuid
);
```

5.3 Table: link_instances

```
CREATE TABLE link_instances (
  id uuid PRIMARY KEY,
  tenant_id uuid NOT NULL,
  workspace_id uuid NOT NULL,
  link_type text NOT NULL,
  from_object_id uuid NOT NULL REFERENCES object_instances(id),
  to_object_id uuid NOT NULL REFERENCES object_instances(id),
  properties jsonb NOT NULL DEFAULT '{}'::jsonb,
  classification text NOT NULL CHECK (
    classification IN ('public', 'internal', 'confidential', 'restricted')
  )
);

ALTER TABLE link_instances ENABLE ROW LEVEL SECURITY;
ALTER TABLE link_instances FORCE ROW LEVEL SECURITY;

CREATE POLICY link_instances_tenant_workspace_select ON link_instances
FOR SELECT
USING (
  tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::uuid
  AND workspace_id = NULLIF(current_setting('app.current_workspace_id', true), '')::uuid
);

CREATE POLICY link_instances_tenant_workspace_write ON link_instances
```



```
FOR INSERT WITH CHECK (
  tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::uuid
  AND workspace_id = NULLIF(current_setting('app.current_workspace_id', true), '')::uuid
);
```

5.4 Table: goal_contracts

```
CREATE TABLE goal_contracts (
  id uuid PRIMARY KEY,
  tenant_id uuid NOT NULL,
  workspace_id uuid NOT NULL,
  objective text NOT NULL,
  constraints jsonb NOT NULL DEFAULT '[]'::jsonb,
  allowed_actions text[] NOT NULL DEFAULT ARRAY[]::text[],
  blocked_actions text[] NOT NULL DEFAULT ARRAY[]::text[],
  acceptance_criteria jsonb NOT NULL DEFAULT '[]'::jsonb,
  risk_class text NOT NULL CHECK (risk_class IN ('low', 'medium', 'high',
    'unacceptable')),
  approval_boundaries text[] NOT NULL DEFAULT ARRAY[]::text[],
  budget jsonb NOT NULL DEFAULT '{}'::jsonb,
  created_by uuid NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

ALTER TABLE goal_contracts ENABLE ROW LEVEL SECURITY;
ALTER TABLE goal_contracts FORCE ROW LEVEL SECURITY;

CREATE POLICY goal_contracts_tenant_workspace_select ON goal_contracts
FOR SELECT
USING (
  tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::uuid
  AND workspace_id = NULLIF(current_setting('app.current_workspace_id', true), '')::uuid
);

CREATE POLICY goal_contracts_tenant_workspace_write ON goal_contracts
FOR INSERT WITH CHECK (
  tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::uuid
  AND workspace_id = NULLIF(current_setting('app.current_workspace_id', true), '')::uuid
);
```

5.5 Table: audit_logs

```
CREATE TABLE audit_logs (
  id uuid PRIMARY KEY,
  tenant_id uuid NOT NULL,
  workspace_id uuid NOT NULL,
  actor_id uuid NOT NULL,
  actor_session_id uuid,
  event_type text NOT NULL,
  resource_type text,
  resource_id uuid,
  oauth_scope_used text,
  tool_called text,
  policy_result text NOT NULL CHECK (
    policy_result IN ('allow', 'deny', 'require_human', 'not_applicable')
  ),
  before_hash text,
  after_hash text,
  previous_event_hash text,
  event_hash text NOT NULL,
  event_payload jsonb NOT NULL DEFAULT '{}'::jsonb,
  created_at timestamptz NOT NULL DEFAULT now()
);

ALTER TABLE audit_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE audit_logs FORCE ROW LEVEL SECURITY;

CREATE POLICY audit_logs_tenant_workspace_select ON audit_logs
FOR SELECT
USING (
  tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::uuid
  AND workspace_id = NULLIF(current_setting('app.current_workspace_id', true), '')::uuid
);

CREATE POLICY audit_logs_append_only_insert ON audit_logs
FOR INSERT WITH CHECK (
  tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::uuid
  AND workspace_id = NULLIF(current_setting('app.current_workspace_id', true), '')::uuid
);
```

Runtime roles must not receive UPDATE or DELETE on audit_logs.

5.6 Classification Propagation Rule Engine

Classification order:

```
classification_order:
  public: 0
  internal: 1
  confidential: 2
  restricted: 3
```

Propagation rule:

```
derived_artifact_classification:
  output_classification: max(parent.classification for parent in source_object_ids)
  downgrade_allowed_only_if_all:
    - actor_has_scope: classification:downgrade
    - human_approval_record_exists: true
    - redaction_review_status: approved
    - audit_event_created: true
```

Database responsibility:

The application may propose classification metadata, but the database-side write path must recompute or validate it before commit. Any derived artifact with `source_object_ids` that reference confidential parents must be at least confidential unless a downgrade approval record is linked.

Applied to agent outputs:

```
Private GitHub issue + repository file
-> extracted Statement
-> candidate Task
-> patch Artifact
-> PR review packet
-> terminal answer chunk
```

All descendants inherit at least the repository classification.

6. THE FUTURE CRYPTOGRAPHIC UPGRADE PATH (ANTI-RED TEAM PREPARATION)

6.1 Launch Stance

Lean verification and ZK proofs are not launch enforcement mechanisms. Launch enforcement is OAuth 2.0 scopes, short-lived JWTs, RLS, sandboxing, runtime assertions, and append-only audit.

Future-proofing is accomplished with durable identifiers and nullable proof fields so future assurance does not require data model replacement.

6.2 Required Data Accommodations

Add dormant fields to policy, permission, and audit records:

```
future_proof_fields:
  spec_rule_ids: string[]
  formal_rule_version: string | null
  lean_spec_id: string | null
  conformance_test_vector_set_id: string | null
  proof_requirement: none | runtime_assertion | lean_ci | zk_attestation
  proof_artifact_id: string | null
  verifier_version: string | null
  verifier_result: pass | fail | not_required
```

Future tables:

```

formal_rules:
  id: string
  rule_name: string
  english_rule: string
  applies_to_tables: string[]
  launch_enforcement: runtime_assertion
  future_enforcement: lean_ci

zk_predicates:
  id: string
  predicate_name: string
  public_input_schema: object
  witness_schema_ref: string
  verifier_version: string
  status: draft | active | deprecated

proof_artifacts:
  id: string
  predicate_id: string
  public_inputs_hash: string
  proof_blob_ref: string
  verifier_result: pass | fail
  linked_permission_check_id: string

```

6.3 JWT Validator To ZK Verifier Transition

Launch mode:

Tool Router verifies JWT signature and claims
 -> checks scopes
 -> sets RLS session context
 -> executes scoped tool

Future high-assurance mode:

Tool Router receives JWT plus optional proof_artifact_id
 -> verifies JWT for session identity
 -> calls Proof Verifier for authority predicate
 -> validates public inputs match tenant_id, workspace_id, action, resource, and policy_version
 -> sets RLS context only if JWT and proof both pass

Authority circuit public inputs:

```

CanPerformAction_public_inputs:
  - tenant_id_hash
  - workspace_id_hash
  - actor_id_hash
  - session_id_hash
  - action_kind
  - resource_id_hash
  - policy_version_hash
  - scope_set_hash
  - expiration_time_bucket

```

Private witnesses remain inside enterprise boundary:

```

CanPerformAction_private_witnesses:
  - role_memberships
  - delegated_token_chain
  - resource_classification
  - policy_rules
  - approval_records
  - revocation_set

```

6.4 RLS To Lean CI Translation

RLS invariant in English:

For every runtime query against object_instances, returned rows must have tenant_id equal to app.current_tenant_id and workspace_id equal to app.current_workspace_id.

Lean CI upgrade target:

```
lean_invariants_to_add_later:
```

TenantIsolation:
 source: object_instances_tenant_workspace_select
 claim: no query returns a row outside runtime tenant/workspace context
ScopeSubsetDelegation:
 source: delegation_token_rules
 claim: child delegation scopes are always subset of parent scopes
ClassificationPropagation:
 source: derived_artifact_classification
 claim: derived output classification is at least as restrictive as every parent
AuditAppendOnly:
 source: audit_logs_permissions
 claim: runtime roles cannot update or delete audit events
WorkflowPolicyDominance:
 source: strategy_selection_order
 claim: Mo0 candidates cannot materialize actions when Atlas policy or workflow guard denies

Lean and ZK are compliance hardening layers. They do not replace OAuth, RLS, sandboxing, or audit.

7. END-TO-END SYSTEM INTEGRATION TRACER (AAA CAMERA BUG CASE)

Scenario:

domain_pack: AAA Game Development Vertical Slice
tenant_id: tenant_game_studio
workspace_id: workspace_game_studio
repository: private/github/org/aaa-vertical-slice
bug_title: Camera clips through wall
target_file: Source/Game/Camera/ThirdPersonCameraController.cs
project_object: project_atlas_aaa_wedge
initial_classification: confidential

7.1 Phase A: Ingestion & Ontology Delta

Step	Operation	Service Owner	Table/JSON Mutation	Policies Evaluated	Token / Latency Profile
A1	Receive GitHub issue webhook	API Gateway	audit_logs(event_type='github.webhook.received')	OAuth client valid, tenant resolved, body size limit	0 tokens / 20-80ms
A2	Fetch issue and file metadata	Data Connector Service	object_instances(object_type='RawExternalRecord')	repo:read, connector allowlist, RLS context	0 tokens / 300ms-2s
A3	Normalize external record	Ingestion Service	object_instances(object_type='NormalizedRecord')	no secret persistence, classification inheritance	Luna 1k in / 300 out / 1-3s
A4	Extract statement	Research Orchestrator	object_instances(object_type='Statement', lifecycle='candidate')	extraction is not acceptance, source refs required	Luna 2k in / 600 out / 2-4s
A5	Insert candidate task	Atlas Object Store	object_instances(object_type='Task', properties.title='Fix camera clipping')	candidate cannot drive next action, RLS write	0 tokens / 30-100ms
A6	Link bug, repo file, build, task, project	Link Store	link_instances(link_type in ['affects', 'derived_from', 'belongs_to'])	link endpoint tenant/workspace match	0 tokens / 40-150ms
A7	Recalculate graph snapshot	Graph Engine	object_instances(object_type='GraphProjection')	classification propagation, async snapshot versioning	0 tokens / 150ms-8s
A8	Close ingestion event	Audit Service	audit_logs(event_type='ingestion.delta.committed')	append-only ledger, event hash chain	0 tokens / 50-200ms

7.2 Phase B: Strategy Selection & Goal Contracting

Step	Operation	Service Owner	Table/JSON Mutation	Policies Evaluated	Token / Latency Profile
B1	Create GoalContract	Goal Contract Engine	goal_contracts(objective='Patch camera clipping bug')	goal create scope, workspace RLS	Terra 1k in / 500 out / 1-2s
B2	Classify task	Task Classification Service	object_instances(object_type='TaskClassification')	code_change risk rules, core camera path detection	Terra 1.5k in / 500 out / 1-3s
B3	Fast-path assessment	Meta-Orchestrator	object_instances(object_type='FastPathAssessment', result='failed')	low-risk fast-path criteria	0 tokens deterministic / <100ms
B4	Score orchestrator registry	Strategy Selection Engine	object_instances(object_type='StrategySelection')	package active, risk limit, required tools	Sol 4k in / 1k out / 4-10s
B5	Start blended strategy	MoO Runtime	meta_orchestration_runs, orchestrator_runs for Coding, Critic, Safety-Verification	runtime budget required, max orchestrator count	0 tokens / 100-300ms

Fast-path failed because:

```
fast_path_rejection:
  touches_core_system: true
  system_area: third_person_camera_controller
  action_kind: code_change
  verification_required: true
  private_repo_external_pr_required: true
```

7.3 Phase C: Execution, Critique, Sandbox Verification, Fatigue Gate

Step	Operation	Service Owner	Table/JSON Mutation	Policies Evaluated	Token / Latency Profile
C1	Generate SYSTEM_DESIGN.md	Planner/Coding Orchestrator	object_instances(object_type='DesignArtifact') and workspace file write through Tool Router	patch:write, SYSTEM_DESIGN required for medium risk code	Sol 6k in / 1.5k out / 10-25s
C2	Issue JWT delegation token	Identity Service	agent_sessions, audit_logs(event_type='delegation_token.issued')	child scopes subset parent, TTL <= 15m	0 tokens / 40-100ms
C3	Read repository file	Tool Router	tool_calls(tool='repo.read_file')	JWT signature, repo:read, path allowlist, RLS	0 tokens / 1-3s
C4	Generate patch candidate	Coding Orchestrator	object_instances(object_type='PatchArtifact', lifecycle='candidate')	classification inheritance from repo file	Sol 22k in / 5.5k out / 35-90s
C5	Apply patch in sandbox	Tool Router	tool_calls(tool='sandbox.apply_patch'), tool_results	patch:write, sandbox:execute, path allowlist	0 tokens / 5-25s
C6	Critic review	Critic Orchestrator	object_instances(object_type='CriticFinding', severity='blocking')	core camera clamping boundary check	Terra 12k in / 2k out / 10-35s
C7	Bounded rework	Coding Orchestrator	PatchArtifact(v2), link_instances(link_type='resolves')	max critic rework rounds = 1	Sol 14k in / 3k out / 25-70s
C8	Run unit and integration tests	Safety-Verification Orchestrator	VerificationRun(status='passed')	test required for core code, sandbox only	Luna 2k in / 700 out + tools / 1-6m
C9	Fatigue filter	Human Approval Orchestrator	object_instances(object_type='ApprovalFilterDecision')	suppress internal alerts, require review-ready packet only	0 tokens / <100ms

Human fatigue result:

```
fatigue_gate_result:
  notifications_suppressed:
    - patch_started
    - sandbox_started
    - critic_objection_created
```

```

- tests_started
final_notification:
  type: review_ready_pr
  reason: private PR needs human review before merge

```

7.4 Phase D: Closure & Audit Anchoring

Step	Operation	Service Owner	Table/JSON Mutation	Policies Evaluated	Token / Latency Profile
D1	Select final action	Meta-Orchestrator	object_instances(object_type='NextActionDecision', action='open_pr')	verified candidate required	Sol 3k in / 700 out / 4-8s
D2	Permission check PR create	Policy Engine	object_instances(object_type='PermissionCheck', decision='allow')	pr:create, private repo only, no merge	0 tokens / 30-120ms
D3	Open private PR	Tool Router	tool_calls(tool='github.open_pr'), object_instances(object_type='PullRequestArtifact')	JWT, pr:create, branch namespace, no protected branch merge	0 tokens / 1-4s
D4	Update task to review	Atlas Action Engine	object_instances(Task.status='review')	task:write, ActionRequiresPolicy, RLS write	0 tokens / 50-200ms
D5	Append audit chain	Audit Service	audit_logs for decision, permission, tool, task update	append-only, hash chain, no agent direct append	0 tokens / 100-500ms
D6	Generate review packet	MoO Runtime	object_instances(object_type='ReviewPacket')	classification inheritance, no raw prompt trace	Luna 2k in / 600 out / 2-5s
D7	Notify engineer	Human Approval Orchestrator	object_instances(object_type='ReviewRequest'), audit_logs(event_type='human.review_requested')	single notification per loop, code owner review required	0 tokens / 0.5-3s

7.5 Tracer Cost And Latency Budget

```

gpt_5_6_route:
  meta_orchestrator: Sol
  coding_orchestrator: Sol
  critic_orchestrator: Terra
  safety_verification: Terra_or_Luna
  packetization: Luna
  explicit_cache_breakpoint:
    contents:
      - goal_contract
      - issue_summary
      - repository_file_summary
      - policy_summary
      - acceptance_criteria
  minimum_cache_life: 30m

```

Estimated loop budget:

Category	Estimate
Logical input tokens	120k-270k depending cache reuse
Output tokens	18k-30k
Cached effective input reduction	25%-40% when shared context reused
Model cost	approximately USD 1.10-1.80 under GPT-5.6 Sol/Terra/Luna routing assumptions
Tool/sandbox time	2-8 minutes
LLM time	2-5 minutes
End-to-end p50	7-11 minutes
End-to-end p95	16-24 minutes
Human notifications	1

7.6 Acceptance Criteria For This PRD

ID	Acceptance Criterion
AC1	Every tool-executed action is traceable to GoalContract -> MetaOrchestrationRun -> OrchestratorRun -> AgentSession -> JWT -> ToolCall -> AuditEvent.
AC2	PostgreSQL RLS blocks cross-tenant and cross-workspace reads/writes even if application code asks for the wrong rows.
AC3	Candidate/generated records cannot drive authoritative next actions until promoted or verified by policy.
AC4	Low-risk fast-path executes only reversible, local, non-sensitive actions.
AC5	Human approval is required for protected branch merge, production deployment, public export, permission change, destructive delete, and secret/key operations.
AC6	Derived artifacts inherit the highest classification of their parent data nodes.
AC7	Lean and ZK metadata fields exist as upgrade hooks but are not launch enforcement dependencies.